

Isaiah Pajarillo, Dylan Suzuki, Jackson Muller, Aidan Hodges

BJ Johnson

CMSI 4072

4/15/26

Extra Credit

Problem 3.10

Write a class that supports course grading; it contains an array of 30 students such that each student is identified by an integer and has a course grade. There is also a method that can change the student grade and a method that computes the grade point average for the entire class of 30 students.

```
class CourseGrading:
    def __init__(self):
        self.students = [{"id": i, "grade": 0} for i in range(1, 31)]

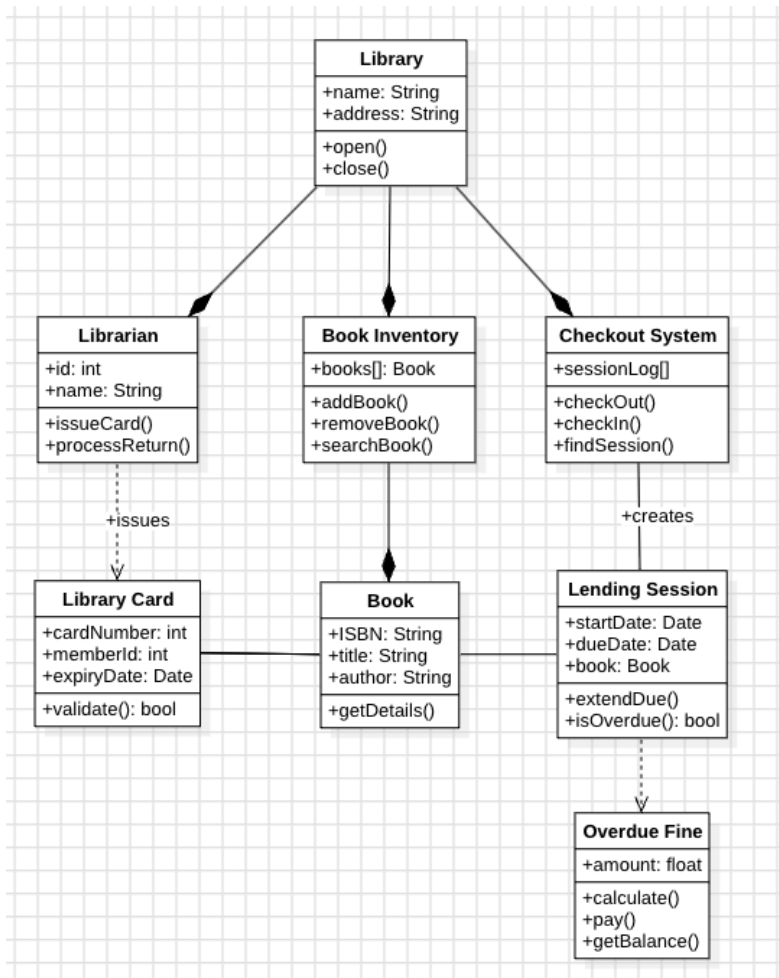
    def change_grade(self, student_id, new_grade):
        for student in self.students:
            if student["id"] == student_id:
                student["grade"] = new_grade
        return

    def grade_point_average(self):
        return sum(s["grade"] for s in self.students) / 30
```

Problem 4.2

Draw a class diagram of a library lending books using the following classes: Librarian, Lending Session, Overdue Fine, Book Inventory, Book, Library, Checkout System, and Library Card.

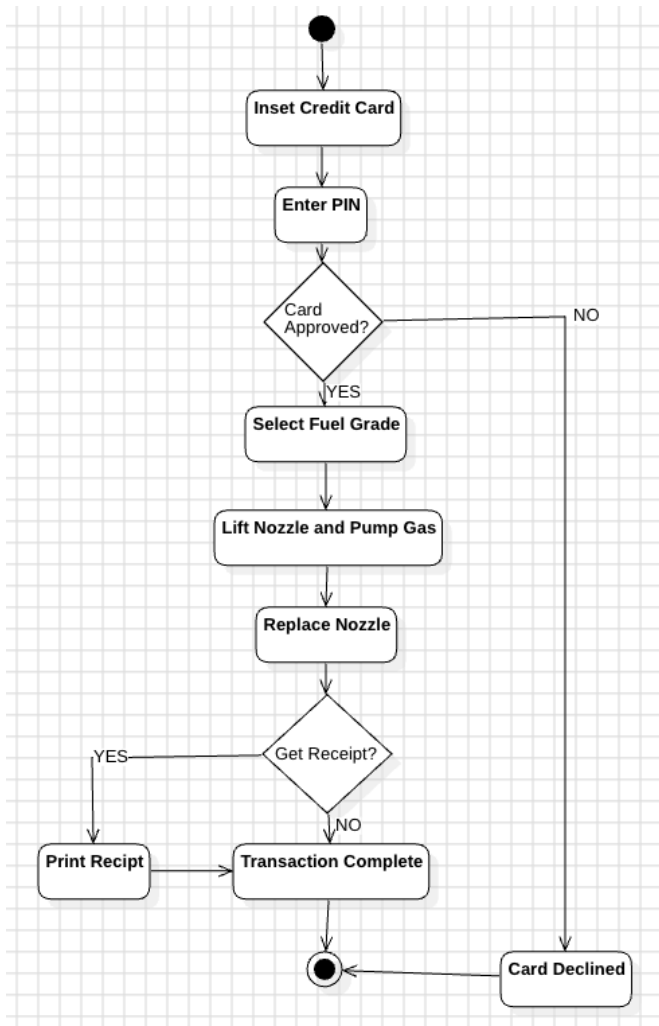
Download and install a UML tool such as StarUML, ArgoUML, or Poseidon and use the tool to draw the class diagram.



Problem 4.3

Draw an activity diagram of pumping gas and paying by credit card at the pump. Include at least five activities, such as "Select fuel grade" and at least two decisions, such as "Get receipt?"

Download and install a UML tool such as StarUML, ArgoUML, or Poseidon and use the tool to draw the diagram. If you did this for the previous problem, you may use the same tool.



Problem 10.6

For the following program design a minimal test suite:

```

public static void main(String args[]) {
    int i,j;
    int k = 0;
    i = Integer.parseInt(args[0]);
    j = Integer.parseInt(args[1]);
    if( i > 30 ) {
        if( i <= 60 && j <= 150 )
            k = 1;
        else if( i <= 90 && j <= 150 )
            k = 2;
        else
            k = 3;
    }
}
  
```

```

    }
    if( i == j && i <= 30 )
        k=4;
    System.out.println( k );
}

```

Test	i	j	Expected k	Reason
T1	25	25	4	$i == j$ and $i \leq 30 \rightarrow k = 4$
T2	15	30	0	$i \leq 30, i \neq j \rightarrow k$ stays 0
T3	50	120	1	$i > 30, i \leq 60, j \leq 150 \rightarrow k = 1$
T4	80	140	2	$i > 30, i > 60, i \leq 90, j \leq 150 \rightarrow k = 1$
T5	100	50	3	$i > 30, i > 90 \rightarrow$ else branch $\rightarrow k = 3$
T6	55	160	3	$i > 30, i \leq 60$ but $j > 150 \rightarrow$ else branch $\rightarrow k = 3$

Problem 10.8

In test driven development, a test is written and then the code to pass it. Given the following test, write a method that passes the test.

```

public class TestGrade {
    Grade testGrade;
    public void testGetFinalGrade() {
        assert( testGrade.getFinalGrade(70) == "Pass" );
        assert( testGrade.getFinalGrade(69) == "Fail" );
    }
};

public class Grade {
    public String getFinalGrade(int score) {
        if (score >= 70)
            return "Pass";
        else
            return "Fail";
    }
}

```

Expand the test in exercise 10.8 to include grades A to F using the following grade scale:

Letter Grade	Minimum Percentage
A	90
B	80
C	70
D	60
F	Less than 60

```
public class TestGrade {
    Grade testGrade;

    public void testGetFinalGrade() {
        assert( testGrade.getFinalGrade(70) == "Pass" );
        assert( testGrade.getFinalGrade(69) == "Fail" );
        assert( testGrade.getLetterGrade(95) == "A" );
        assert( testGrade.getLetterGrade(85) == "B" );
        assert( testGrade.getLetterGrade(75) == "C" );
        assert( testGrade.getLetterGrade(65) == "D" );
        assert( testGrade.getLetterGrade(59) == "F" );
    }
};

public class Grade {
    public String getFinalGrade(int score) {
        if (score >= 70)
            return "Pass";
        else
            return "Fail";
    }

    public String getLetterGrade(int score) {
        if (score >= 90)
            return "A";
        else if (score >= 80)
            return "B";
        else if (score >= 70)
            return "C";
        else if (score >= 60)
            return "D";
        else
            return "F";
    }
}
```

}

Problem 11.2

Ambiguities

1. “when clouds of black smoke start to come off” vague
2. “Slice the lemon into paper-thin slices and add” lemon already used
3. “coat the mushrooms in flour” mushrooms not listed
4. “add the sugar” sugar not listed
5. “kill it by sprinkling it with frobs” confusing, what is frobs?

Omissions

1. Mushrooms not listed in ingredients
2. Sugar not listed in ingredients
3. Garlic never used
4. Parmesan quantity not specified
5. No cooking time for pockwester
6. No milk and flour amounts listed
7. Frozen lemon juice not used

Contradictions

1. Instruction calls for 2 cups of oj but only 1 cup listed in ingredients
2. Lemon frozen the night before, then the recipe tells us to slice the lemon
3. Eggs are told to be stirred after it was already added